Do graphs make credit models better?

A leakage-safe test of graph features, graph neural networks and hybrids on four public credit datasets

4	public credit datasets
26	graph models vs XGBoost
5	seeds, paired bootstrap CIs

- One dataset gave a large, clean win. Three gave nothing. This talk is about why.

THE QUESTION

What a credit desk actually asks

Not “is the AUC higher?” but “does the lift survive a fair test, well enough to justify a pilot?”

Ranking

Test AUC and PR-AUC on a frozen split, against the same XGBoost on the same rows.

Calibration

Probabilities recalibrated on validation (isotonic) and checked on test.

Stability

Population stability index across cohorts. A score that drifts is a score you cannot use.

Business

Approval rate at a fixed bad rate: how many more people can we lend to at the same risk?

Pre-registered pass mark

A model family counts as lift only if AUC and PR-AUC both beat XGBoost with a 95% CI that excludes zero, on at least two datasets, without losing calibration or stability.

- We wrote the pass mark down before running anything. That is what makes a negative result worth hearing.

THE LADDER

Five tiers, from a SQL query to a neural network

Each tier asks more of the graph, and more of the serving stack that has to run it.

			Needs at scoring time	Notebook
0	**Tabular**	Logistic regression, XGBoost, MLP	No graph	02
1	**Graph features → XGBoost**	Degree, neighbour default rate, neighbour means	Precomputed lookups	04
2	**Graph neural network**	GraphSAGE, GAT, RGCN	Graph at inference	05
3	**Hybrid**	GNN embedding appended to XGBoost features	Cached embeddings	06
4	**Distilled student**	XGBoost trained on a graph teacher's soft labels	Tabular only	07

- Today: notebook 03 builds the graphs, then 04, 05 and 06 try three ways to use them.

Four datasets, one real graph

Only DGraph-Fin arrives with borrower-to-borrower edges. For the other three, we decide what an edge is.

OBSERVED

DGraph-Fin

1.2M

labelled users

● Real contact graph

3.7M people, 4.3M emergency-contact links they listed themselves

Home Credit

307K

loans

● Own credit records

Each customer linked to their own bureau history. No customer touches another

INVENTED BY US

Lending Club

1.35M

loans

● Invented edges

Same zip or employer, and nearest neighbours. Tested on 2017–18 loans

Taiwan

30K

card holders

● Invented edges

Nearest neighbours only. No real structure exists

● Whether the edges are observed or invented turns out to be the variable that matters.

DGraph-Fin: Huang et al., NeurIPS 2022 Datasets & Benchmarks · Home Credit and Lending Club via Kaggle · Taiwan via UCI

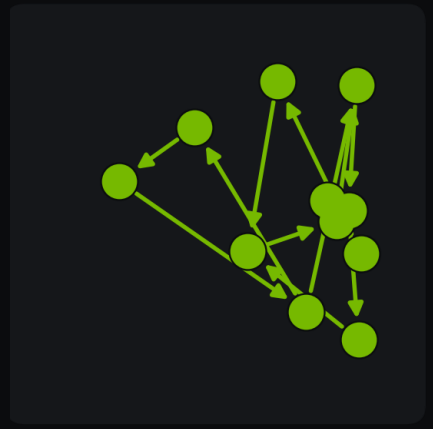
Five ways to give a credit model a graph

Panels three to five are drawn by running the repository's own graph builders on toy data.

OBSERVED

INVENTED BY US

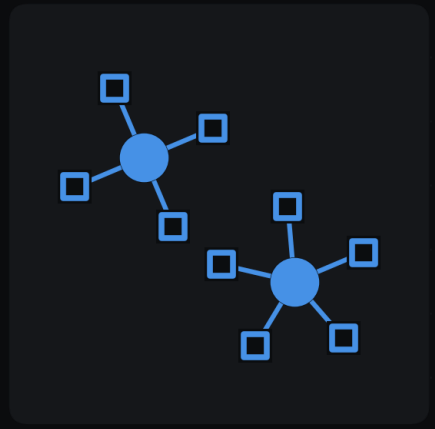
Native



Contacts people actually listed

DGraph-Fin

Relational



Each customer's own credit records

Home Credit

Shared attribute



Same zip code or employer

Lending Club

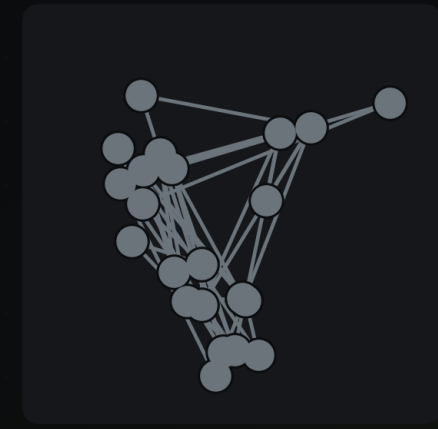
kNN similarity



Most similar applicants

3 datasets

Random control



kNN edges with partners shuffled

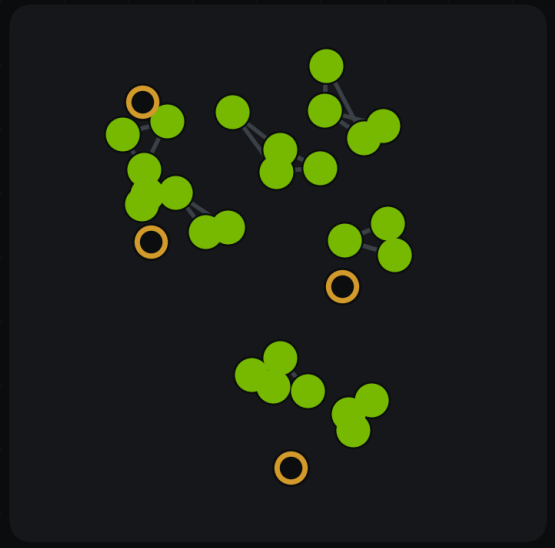
the control group

- The fifth builder is not a model input. It is the experiment's control group.

The clean-room rule

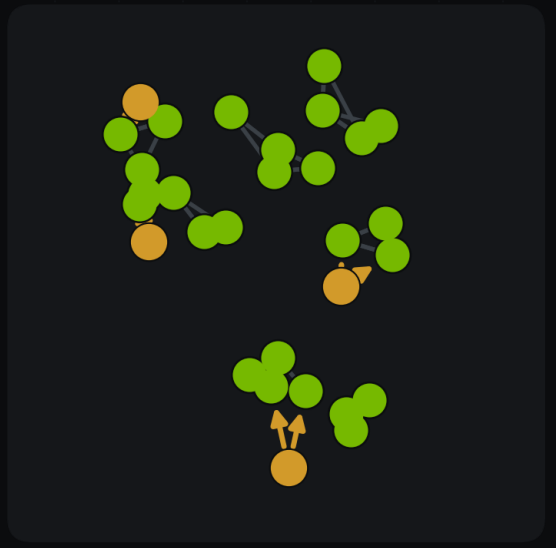
A graph can smuggle test answers into training. So new applicants only join the graph at scoring time.

1 Train



Graph built from past customers only. New applicants (○) are left out.

2 Score



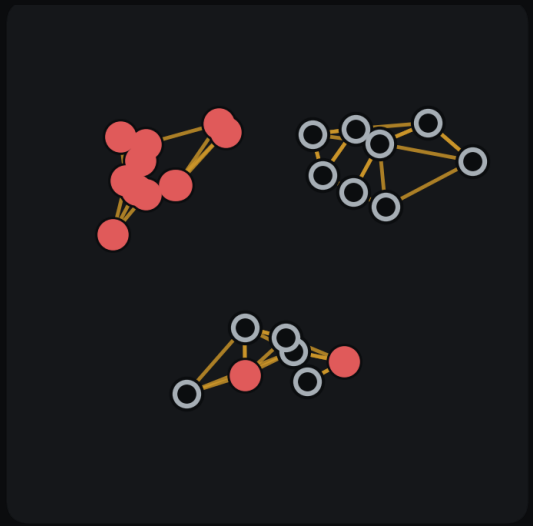
Each new applicant looks up its nearest past customers (→).

- **Past customers**
  their repayment labels are used
- **New applicants**
  label never read, never linked to each other
- **If a test row leaks in**
  an assertion fails the run

- A new applicant is treated exactly as a bank would treat one: looked up, never trained on.

Keep the wiring, destroy the meaning

The random control shuffles every edge's partner. If a model does as well on that, structure was never the reason.



defaulters link to defaulters



same number of links, random partners

● defaulted
○ repaid

Same degrees.

No meaning.



What it keeps

Every applicant keeps the same number of links. Same graph size, same density.

What it destroys

Who is linked to whom. Defaulters stop clustering: 1.9× the base rate falls to 0.9×.

One honest caveat

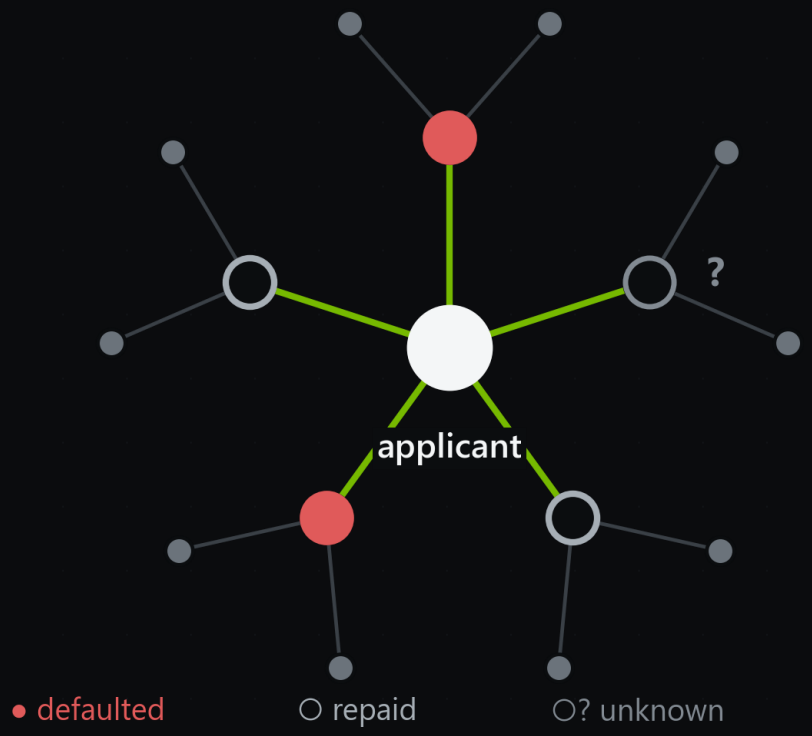
Models read links both ways. Shuffling breaks mutual pairs, so the control is about a third denser.

● Every kNN result in this talk is reported next to its shuffled twin.

Illustration on synthetic data built with the repository's KnnIndex and rewire(). Not a benchmark result.

Graph features a bank can compute in SQL

Summarise each applicant's contacts into a few extra columns, then train the same XGBoost.



Neighbour default rate
2 of 4 known contacts defaulted → 50%

Degree
5 contacts listed

Contact-type counts
how many of each of 12 relationship types

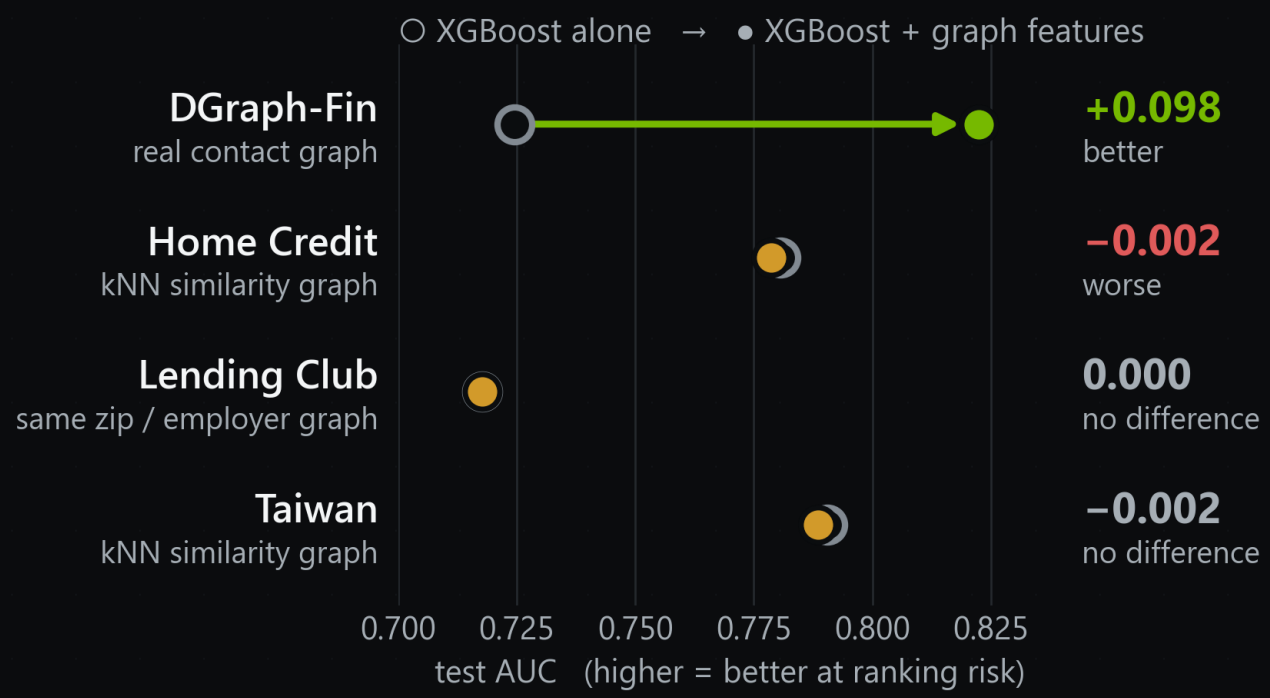
Two-hop reach
how many people the contacts know

Applicant vs contacts
own income minus the contacts' average, and so on

The unknown contact (?) is ignored: default rates only ever use outcomes from the training period.

Real edges: a large win. Invented edges: nothing

Each row: XGBoost alone, then the same XGBoost with graph features added. Same test rows, five seeds.



Real contacts

AUC 0.725 → 0.822 on DGraph-Fin. Significant on all five seeds, the largest gain in the study.

Invented edges

On the other three datasets the dots sit on top of each other: within 0.002 of XGBoost.

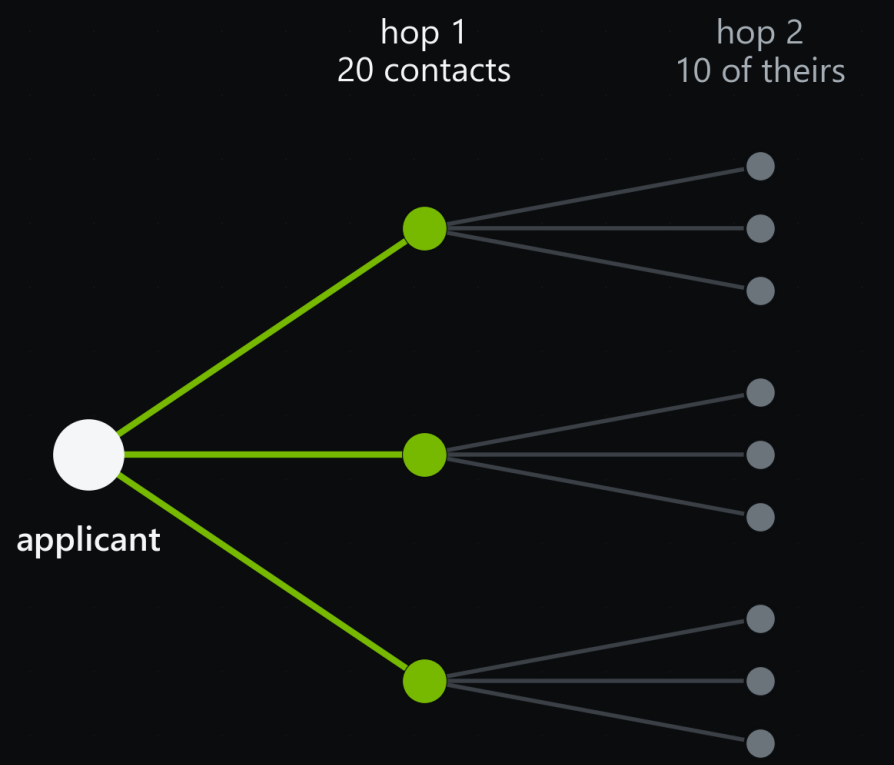
The control agrees

kNN features never beat the same features computed on the shuffled graph.

● Hand-built graph features add almost ten AUC points, but only where the relationships are real.

A GNN on 3.7 million people

Each layer reaches one hop further. After two, an applicant has seen its contacts' contacts.



Applicant + sampled contacts

↓

GraphSAGE layer 1 · 128

↓

GraphSAGE layer 2 · 64 = embedding

↓

Probability of default

Scale

Sample 20 contacts, then 10 of theirs, so memory stays flat on the full graph.

Imbalance

About 1.3% defaulters. Weighted loss, early stopping on validation AUC.

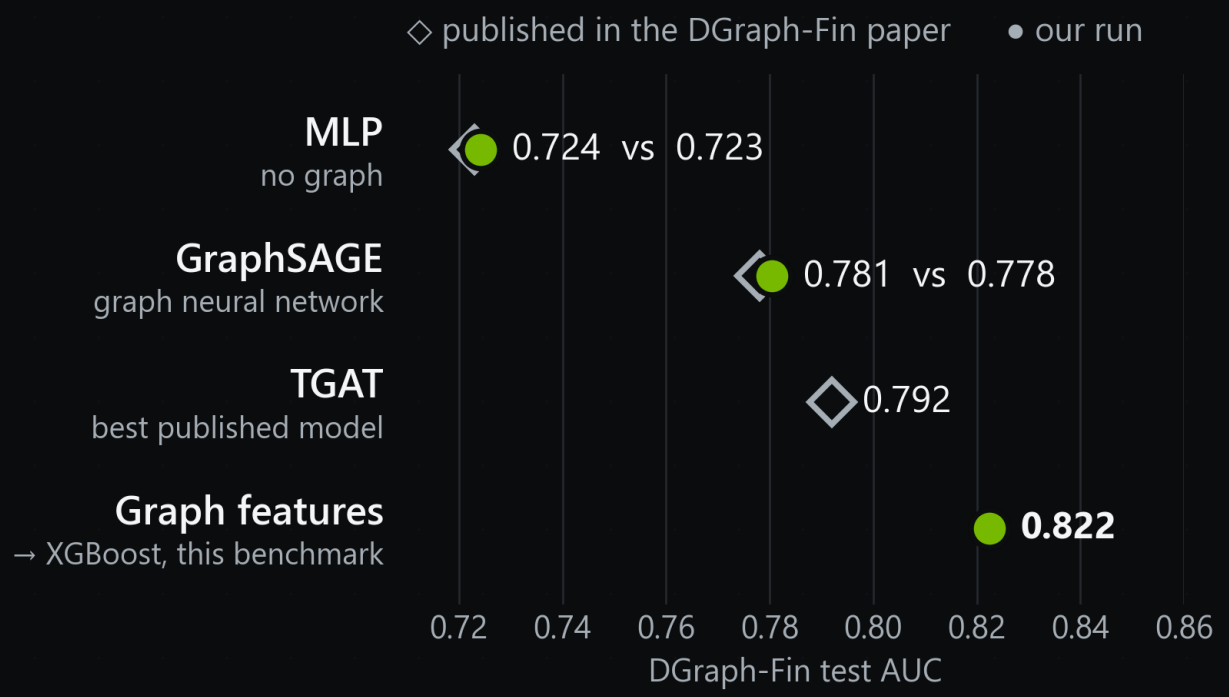
Reused later

The 64-number embedding is saved per applicant and fed to the hybrid.

- The embedding is a 64-number summary of everything within two hops of the applicant.

First, prove the harness is honest

Before showing GNNs losing, we reproduced the published DGraph-Fin results on the official split.



Reproduced

Our MLP and GraphSAGE land within 0.003 AUC of the paper. The pre-registered check passes.

Then beaten

Plain graph features into XGBoost score 0.822, above the best published model, TGAT, at 0.792.

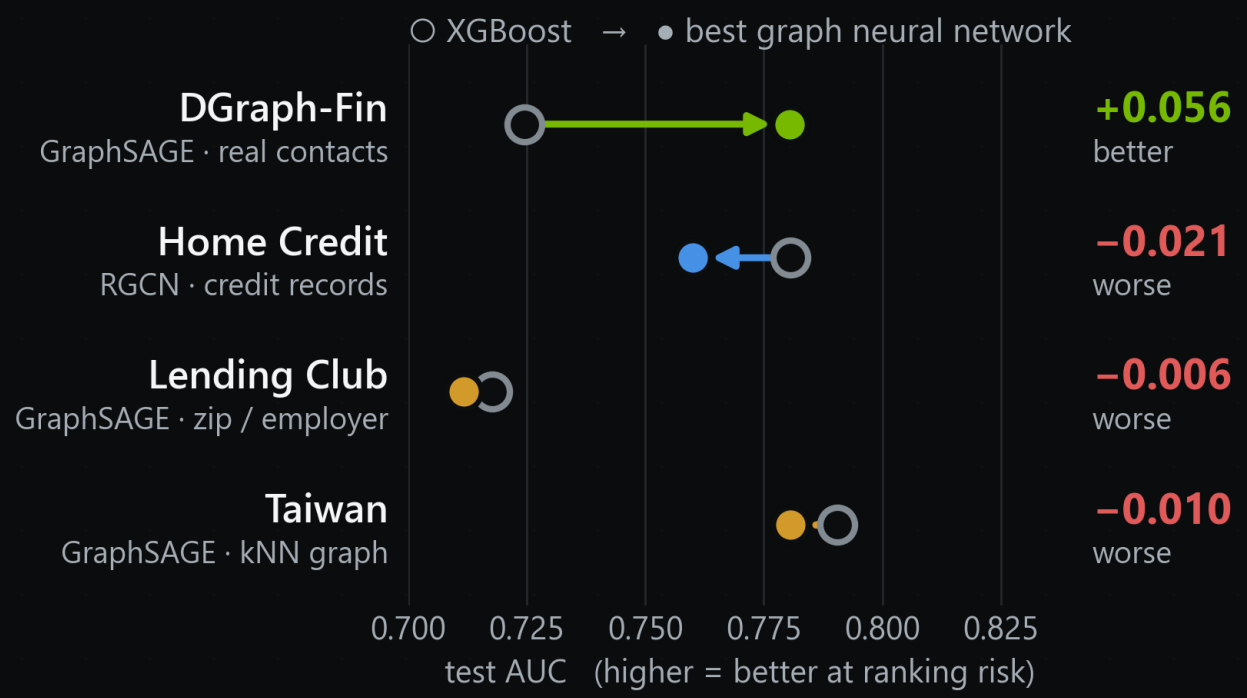
Why it matters

When the GNNs lose on other datasets, it is not because the training code is broken.

● A benchmark that cannot reproduce the paper cannot be trusted to contradict it.

GNNs win on real edges and lose everywhere else

Same chart as before: XGBoost, then the best graph neural network for each dataset.



Real contacts

GraphSAGE adds 0.056 AUC on DGraph-Fin. Better on every seed.

Real records

On Home Credit the RGCN loses 0.021. XGBoost already sees that history through engineered columns.

Invented edges

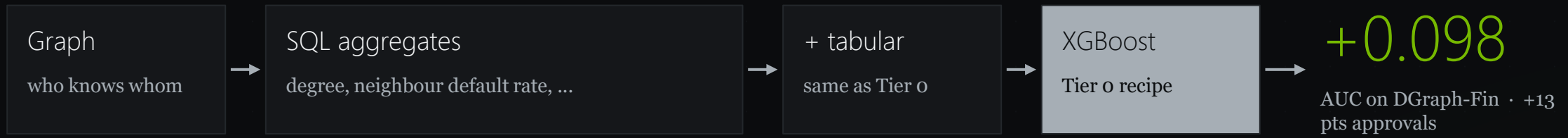
Even the best GNN loses on Lending Club and Taiwan. The arrows point backwards.

Every result here is significant: reliably better on real edges, reliably worse on everything else.

Two ways to put a graph into XGBoost

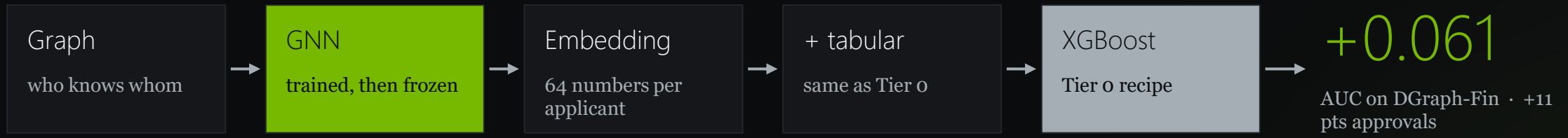
In both, the decision engine stays a gradient-boosted tree. What differs is who builds the graph columns.

TIER 1 · NOTEBOOK 04



No neural network. Scoring needs only a neighbour lookup.

TIER 3 · NOTEBOOK 06

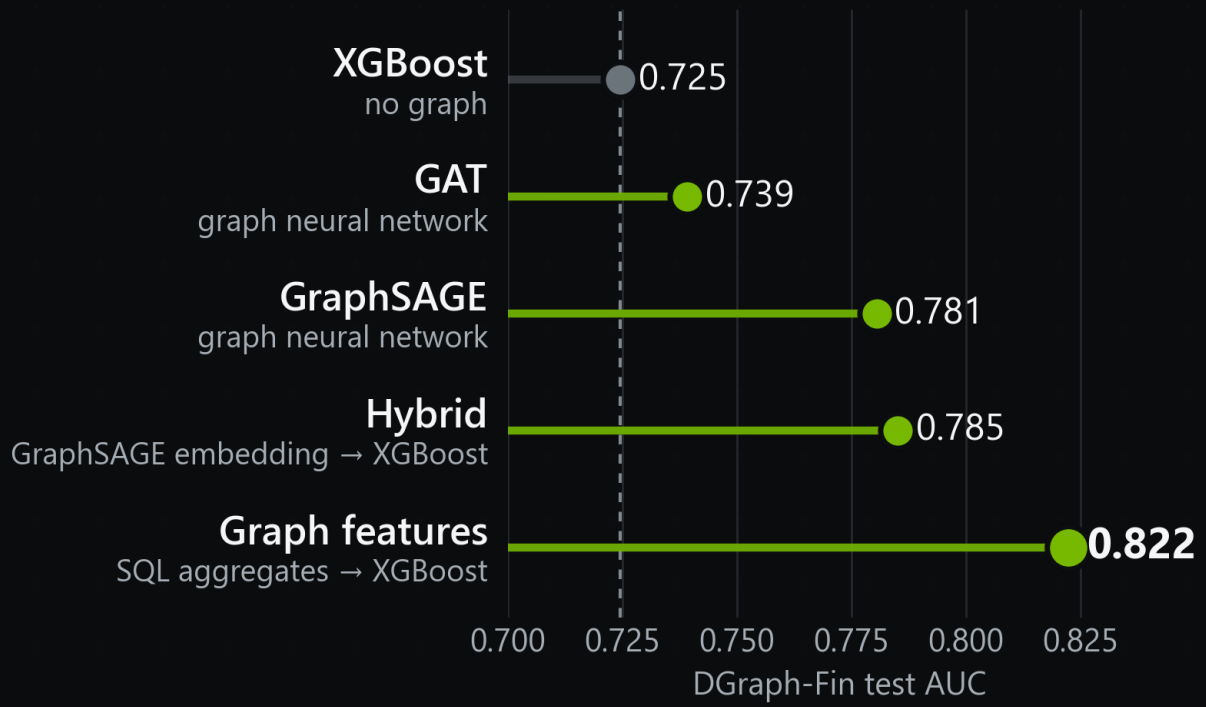


The graph is needed only to refresh embeddings, in batch.

- The hybrid is the commercial candidate: the graph enters as a feature block, the governance stays the same.

The hybrid helps, but SQL features go further

Every model on DGraph-Fin, the one dataset with real edges. Test AUC, mean of five seeds.



The hybrid helps the GNN

Feeding the embedding to XGBoost lifts GAT 0.739 → 0.773 and GraphSAGE 0.781 → 0.785.

SQL features go further

0.822 with no neural network anywhere, and nothing new to serve.

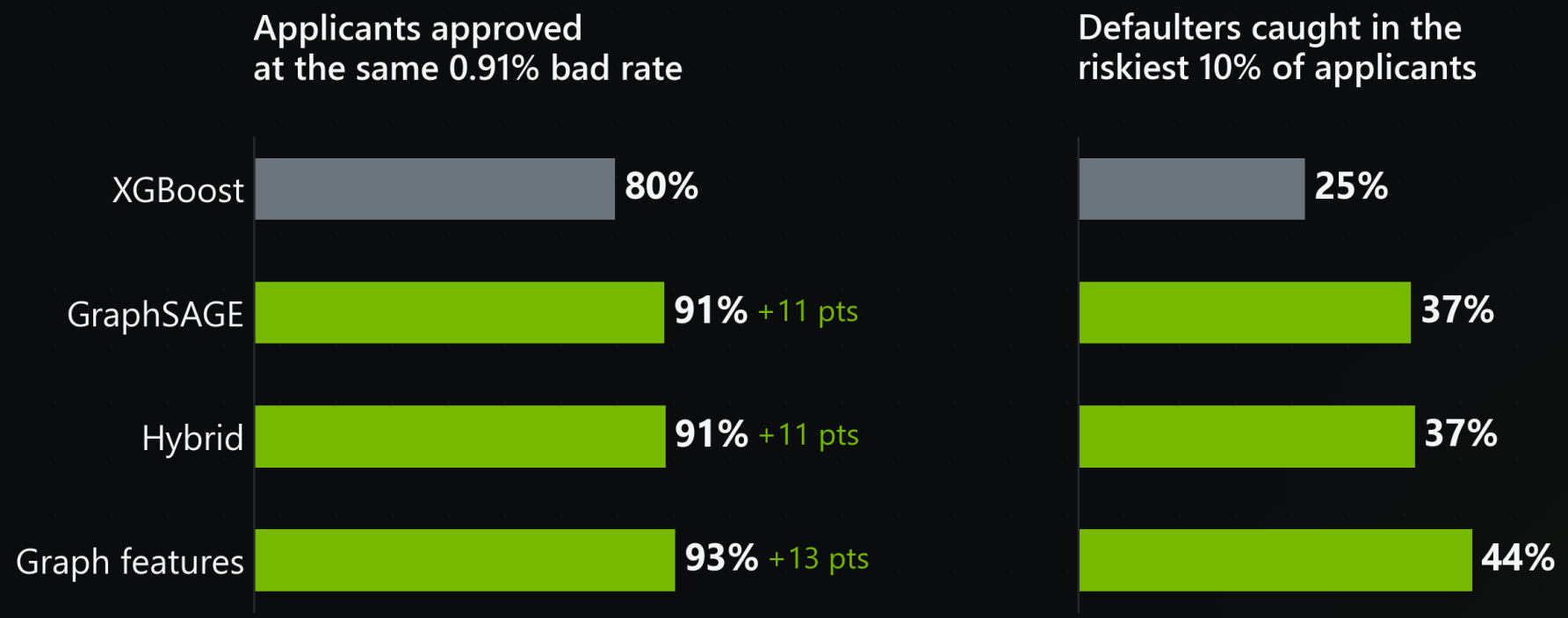
Elsewhere

On the other three datasets the hybrid never gets above XGBoost.

Does the lift need a neural network? On this evidence, no.

What the lift is worth at the credit desk

Hold XGBoost's 0.91% bad rate fixed and ask: how many applicants can each model approve?



13 more applicants in every 100 approved, at the same risk, from features you can compute in SQL.

Stage 08, DGraph-Fin test split, mean of 5 seeds. Labels are a public repayment proxy, not a calibrated PD for a real portfolio.

Graphs only help where the edges are real

The best model of each kind, minus XGBoost, on each dataset.

	Graph features → XGBoost	Graph neural network	Hybrid GNN → XGBoost
DGraph-Fin real contacts	**+0.098** better	**+0.056** better	**+0.061** better
Home Credit own credit records	**−0.002** worse	**−0.021** worse	**−0.015** worse
Lending Club invented edges	**0.000** no difference	**−0.006** worse	**−0.001** worse
Taiwan invented edges	**−0.002** no difference	**−0.010** worse	**−0.005** no difference

3 / 3 families beat XGBoost on real contacts

0 / 9 beat it anywhere else

1 of 4 datasets show lift; the pass mark was 2

The signal is in relationships you observe, not relationships you infer.

Four rules before you build a credit GNN

1 Find the real edges first

Contacts, guarantors, shared devices, payment flows. Observed relationships carried all of the lift here.

2 Start with SQL features

Neighbour default rate and degree beat every GNN: +0.098 AUC against +0.061, with nothing new to serve.

3 Always run a random control

kNN graphs never beat their shuffled twins. Without the control, smoothing looks like structure.

4 Build graphs in a clean room

Test rows attach by lookup, labels come from training only, and an assertion enforces it.

- Gate A verdict: pilot graph features only where real relationship data exists, and start with Tier 1.